

Guardian — Documentation technique du backend

Documentation technique complète — architecture, packages, flux, tests, audit de sécurité

Généré le 2026-09-13. Couvre l'état du backend à la fin de la phase finale (MongoDB + Better Auth + historique d'audits). Le frontend n'a aucun code métier à ce stade.

1. Vue d'ensemble

Guardian est une plateforme d'audit de sécurité assistée par IA. Un utilisateur authentifié envoie un ou plusieurs fichiers (code source, IaC, configs serveur, pipelines CI/CD, fichiers d'environnement) ; le backend les valide, les envoie à Google Gemini Flash pour analyse, sauvegarde le rapport structuré en base, et le restitue au format JSON.

Le backend est un monolithe modulaire NestJS (TypeScript, ESM natif), organisé en modules à responsabilité unique : ingestion de fichiers, intégration IA, authentification, persistance, et durcissement transversal (sécurité HTTP, rate-limiting, gestion d'erreurs).

1.1 Principe directeur adopté pendant le développement

À chaque étape, la priorité a été : **choisir la version la plus récente et stable des outils**, câbler les choses de façon idiomatique à NestJS (injection de dépendances plutôt que valeurs statiques), et **vérifier chaque fonctionnalité par un test réel** (appel HTTP en direct, pas seulement des tests automatisés) avant de la considérer terminée.

2. Stack technique et rôle de chaque package

2.1 Cœur du framework

Package	Rôle
@nestjs/core , @nestjs/common	Le framework NestJS lui-même — DI, décorateurs, cycle de vie des modules
@nestjs/platform-express	Adaptateur HTTP (Express sous le capot) — gère aussi l'upload de fichiers (<code>multer</code>)
@nestjs/config	Charge <code>.env</code> dans <code>process.env</code> , expose <code>ConfigService</code>
@nestjs/swagger	Génère la documentation OpenAPI interactive sur <code>/api/docs</code>
reflect-metadata	Requis par les décorateurs TypeScript (métadonnées de type à l'exécution)

Package	Rôle
<code>rxjs</code>	Programmation réactive — utilisé pour le timeout/retry des appels Gemini

2.2 Sécurité et validation

Package	Rôle
<code>helmet</code>	Ajoute les en-têtes HTTP de sécurité (CSP, HSTS, X-Frame-Options, etc.)
<code>express-rate-limit</code>	Limite le nombre de requêtes par IP — un limiteur global + un limiteur dédié à <code>/audit</code>
<code>class-validator</code>	Valide les DTOs entrants (et la sortie JSON de Gemini) via des décorateurs (<code>@IsString</code> , <code>@IsEnum</code> , etc.)
<code>class-transformer</code>	Transforme un objet JSON brut en instance de classe typée (nécessaire pour que <code>class-validator</code> fonctionne)
<code>multer</code>	Parseur <code>multipart/form-data</code> — gère l'upload de fichiers en mémoire (jamais écrit sur disque)

2.3 IA — Gemini Flash

Package	Rôle
<code>@nestjs/axios</code> , <code>axios</code>	Client HTTP utilisé pour appeler l'API REST Gemini directement (pas de SDK officiel installé — voir §6.2 pour la justification)

2.4 Base de données et authentification

Package	Rôle
<code>mongoose</code>	ODM MongoDB — définit les schémas (<code>AuditReportEntity</code>) et exécute les requêtes
<code>@nestjs/mongoose</code>	Intègre Mongoose dans le système de DI de NestJS
<code>mongodb</code>	Driver natif MongoDB — utilisé directement par Better Auth (voir §6.4)
<code>better-auth</code>	Framework d'authentification complet (inscription, connexion, sessions) — gère ses propres collections Mongo

2.5 Outillage (dev)

Package	Rôle
<code>vitest</code> + <code>@vitest/coverage-v8</code>	Test runner (remplace Jest — voir §6.1)
<code>supertest</code>	Requêtes HTTP simulées contre l'application Nest pour les tests d'intégration (e2e)

Package	Rôle
oxlint	Lintier (remplace ESLint dans le scaffold NestJS le plus récent)
@nestjs/cli , @nestjs/schematics , @nestjs/testing	Outillage NestJS standard
typescript	Compilateur

3. Structure des dossiers et rôle de chaque fichier

```

backend/
+-- .env / .env.example      Variables d'environnement (voir §8)
+-- src/
|   +-- main.ts              Point d'entrée – gère le mode cluster multi-cœurs
|   +-- bootstrap.ts         Construction de l'app Nest : Better Auth, sécurité HTTP, Swagger
|   +-- app.module.ts         Module racine – assemble tous les autres modules
|   +-- app.controller.ts     Route GET / (simple "Hello World", healthcheck implicite)
|   +-- app.service.ts        Logique triviale derrière app.controller.ts
|   |
|   +-- config/
|   |   +-- app-config.service.ts  Accès typé et centralisé à toutes les variables d'env
|   |   +-- app-config.module.ts  Module global exposant AppConfigService partout
|   |
|   +-- database/
|   |   +-- database.module.ts     Connexion Mongoose (MongoDB)
|   |   +-- schemas/
|   |       +-- audit-report.schema.ts  Schéma Mongoose d'un rapport d'audit
|   |       +-- audit-finding.schema.ts  Sous-document d'une vulnérabilité trouvée
|   |
|   +-- auth/
|   |   +-- auth.instance.ts        Construit l'instance Better Auth (config, plugins)
|   |   +-- auth.module.ts          Fournit l'instance Better Auth + le guard via DI
|   |   +-- auth.guard.ts           Vérifie la session sur les routes protégées
|   |   +-- auth.constants.ts       Jeton d'injection (Symbol) pour l'instance Better Auth
|   |   +-- auth.types.ts           Type TypeScript de l'utilisateur authentifié
|   |   +-- current-user.decorator.ts Décorateur @CurrentUser() pour les contrôleurs
|   |
|   +-- gemini/
|   |   +-- gemini.service.ts        Appelle l'API Gemini, gère retry/timeout, valide la réponse
|   |   +-- gemini.module.ts         Câblage NestJS du service
|   |   +-- gemini.constants.ts      Prompt système + schéma JSON strict exigé de l'IA
|   |   +-- gemini.types.ts          Types TypeScript de la requête/réponse REST Gemini
|   |   +-- dto/gemini-analysis-result.dto.ts  Forme validée de ce qu'on accepte de Gemini
|   |
|   +-- audit/
|   |   +-- audit.controller.ts       Endpoints HTTP : POST /audit, GET /audit/history, GET /audit/:id
|   |   +-- audit.service.ts          Orchestration : validation → Gemini → sauvegarde → lecture
|   |   +-- audit.module.ts           Câblage : Mongoose, Multer, rate-limiter dédié
|   |   +-- audit-rate-limit.middleware.ts  Limiteur de requêtes spécifique à /audit
|   |   +-- file-sanitizer.util.ts     Fonctions pures : anti path-traversal, détection binaire/UTF-8
|   |   +-- audit.types.ts            Type interne NormalizedFile
|   |   +-- dto/
|   |       +-- audit-finding.dto.ts   Une vulnérabilité (sévérité, fichier, ligne, remédiation...)
|   |       +-- audit-report.dto.ts    Le rapport complet renvoyé par POST /audit et GET /audit/:id
|   |       +-- audit-history-item.dto.ts  Résumé léger utilisé dans la liste paginée
|   |       +-- audit-history-query.dto.ts Paramètres page/limit de GET /audit/history
|   |       +-- paginated-audit-history.dto.ts  Enveloppe de pagination

```

```

| |      +-- severity.enum.ts          INFO | LOW | MEDIUM | HIGH | CRITICAL
| |      +-- risk-level.enum.ts        LOW | MEDIUM | HIGH | CRITICAL
| |
| +-- common/
|   +-- filters/global-exception.filter.ts  Capture toutes les erreurs, réponse JSON uniforme
|   +-- global-rate-limit.middleware.ts     Limiteur appliqué à toutes les routes
|   +-- rate-limit-response.util.ts        Corps JSON cohérent sur les réponses 429
|
+-- test/
  +-- test-env.ts          Active les tests e2e liés à MongoDB seulement si TEST_MONGODB_URI existe
  +-- app.e2e-spec.ts      Test e2e trivial (GET /)
  +-- audit.e2e-spec.ts    Suite e2e complète : auth, upload, historique, sécurité, rate-limit

```

Chaque fichier `*.spec.ts` à côté d'un fichier source est son test unitaire (convention Vitest/NestJS).

4. Comment tout s'articule

4.1 Séquence de démarrage (`main.ts` → `bootstrap.ts`)

1. `main.ts` décide s'il faut activer le **mode cluster** (`CLUSTERING_ENABLED=true`) : si oui, le processus primaire forke un worker Node par cœur CPU (module natif `cluster`), chacun exécutant sa propre instance de l'app derrière le même port. Si non (par défaut en dev), un seul processus démarre directement.
2. `bootstrap.ts` construit l'application NestJS avec le body-parser désactivé au départ (`bodyParser: false`) — nécessaire car Better Auth a besoin du corps de requête brut, non parsé.
3. Le handler HTTP natif de Better Auth est monté sur `/api/auth/*` **avant** que les parseurs JSON/urlencoded ne soient réactivés manuellement.
4. Helmet et CORS (restreint aux origines de `CORS_ALLOWED_ORIGINS`) sont appliqués.
5. Swagger est généré et exposé sur `/api/docs` .
6. Le serveur écoute sur le port configuré.

4.2 Le graphe de modules (`app.module.ts`)

```

AppModule
+- ConfigModule (charge .env)
+- AppConfigModule (@Global – expose AppConfigService partout)
+- DatabaseModule (connexion Mongoose)
+- AuthModule (@Global – expose l'instance Better Auth + AuthGuard partout)
+- AuditModule
  +- GeminiModule
  +- MongooseModule.forFeature (schéma AuditReportEntity)
  +- MulterModule.registerAsync (limites d'upload lues via DI)

```

Trois éléments transversaux sont enregistrés comme **providers globaux** dans `AppModule` plutôt que câblés dans `bootstrap.ts` : le filtre d'exception (`APP_FILTER`), le pipe de validation (`APP_PIPE`), et le rate-limiter global (middleware appliqué à `'*'`). Ce choix garantit qu'ils sont actifs identiquement en production **et** dans les tests d'intégration, sans dupliquer la configuration.

4.3 Cycle de vie d'une requête `POST /audit`

1. `GlobalRateLimitMiddleware` puis `AuditRateLimitMiddleware` (limite dédiée, plus stricte) vérifient le quota.

2. `AuthGuard` appelle `auth.api.getSession()` avec les en-têtes de la requête ; sans session valide → 401 . L'utilisateur authentifié est attaché à `request.user` .
3. L'intercepteur `FilesInterceptor` (Multer) parse le `multipart/form-data` et charge chaque fichier en **mémoire** (jamais sur disque).
4. `AuditController.audit()` délègue à `AuditService.runAudit()` avec les fichiers et l'id utilisateur (via `@CurrentUser()`).
5. `AuditService.validateAndNormalizeFiles()` vérifie : nombre de fichiers, taille par fichier, taille totale, chemin sûr, contenu non-binaire, encodage UTF-8 valide. Le moindre fichier invalide fait échouer toute la requête avec un message explicite.
6. `GeminiService.analyze()` construit le prompt (système + un bloc par fichier, clairement délimité), appelle l'API REST Gemini avec `responseSchema` pour forcer une sortie JSON strictement structurée, gère timeout/retry, puis **revalide** la réponse de l'IA avec `class-validator` avant de la faire confiance.
7. Le rapport validé est sauvegardé dans MongoDB (`AuditReportEntity`), lié à l'utilisateur.
8. Le rapport complet (avec son `id` MongoDB) est renvoyé au client en 200 OK .

Toute exception à n'importe quelle étape traverse le `GlobalExceptionHandler` , qui uniformise la réponse en `{statusCode, error, message, timestamp, path}` — sans jamais exposer de stack trace ou de détail interne.

4.4 Flux d'authentification

Better Auth gère lui-même ses routes (`/api/auth/sign-up/email` , `/api/auth/sign-in/email` , etc.) et **ses propres collections MongoDB**, via le driver natif `mongodb` (pas Mongoose). Le backend réutilise la même connexion Mongoose sous-jacente pour éviter deux pools de connexions séparés.

Le plugin `bearer` permet au frontend d'envoyer `Authorization: Bearer <token>` plutôt que de dépendre uniquement des cookies cross-origin — plus robuste pour un frontend et un backend sur des origines séparées.

5. Stratégie de tests

56 tests au total, tous automatisés avec Vitest.

5.1 Tests unitaires (42) — aucune infrastructure externe requise

Chaque service est testé en isolation : les dépendances externes (Gemini, MongoDB, Better Auth) sont **mockées** directement (pas de framework de mock magique — de simples objets `vi.fn()` passés au constructeur). Couvre : validation de fichiers, sanitization de chemins, logique de retry/timeout Gemini, filtre d'exception, garde d'authentification.

5.2 Tests d'intégration / e2e (14) — contre une vraie base MongoDB

Utilisent `supertest` contre une instance Nest complète (`AppModule` réellement démarré, avec tous les middlewares/guards/filtres actifs). Seul `GeminiService` est mocké (pour éviter de consommer le quota IA à chaque run) — **l'inscription, l'authentification et la persistance sont 100% réelles**, contre le cluster MongoDB de test.

Ces tests sont **automatiquement ignorés** (`describe.skipIf`) si aucune variable `TEST_MONGODB_URI` n'est fournie à l'environnement d'exécution — pour que `pnpm test:e2e` ne casse jamais dans un environnement sans base de données configurée (ex : premier clone du repo).

Pour les exécuter réellement :

```
TEST_MONGODB_URI="<votre_uri_de_test>" pnpm test:e2e
```

6. Décisions techniques notables (et pourquoi)

6.1 Vitest plutôt que Jest

Le scaffold NestJS le plus récent utilise Vitest par défaut car le projet est en ESM natif (`"type": "module"` , résolution `NodeNext`). Vitest gère l'ESM nativement ; forcer Jest aurait demandé une configuration ESM fragile et non recommandée par l'équipe NestJS elle-même pour ce type de projet.

6.2 Appel REST direct à Gemini plutôt qu'un SDK

Choix délibéré : un appel HTTP brut via `@nestjs/axios` donne un contrôle total sur le timeout, le nombre de tentatives, et la logique de retry sélective (uniquement sur erreurs réseau/429/5xx, jamais sur 4xx) — plus prévisible qu'un comportement de SDK tiers potentiellement opaque.

6.3 Upload de fichiers 100% en mémoire

`multer` est configuré en `memoryStorage()` — **aucun fichier uploadé n'est jamais écrit sur le disque du serveur**. Ceci élimine une classe entière de vulnérabilités (path traversal menant à une écriture de fichier arbitraire, exécution de code via un fichier uploadé, etc.) à la racine, sans dépendre uniquement de la validation applicative.

6.4 Better Auth : driver natif MongoDB, pas Mongoose

Better Auth gère la structure de ses propres collections (utilisateurs, sessions, comptes) en interne. Lui imposer des schémas Mongoose créerait un risque de dérive de schéma avec ses futures migrations internes. Le driver natif est réutilisé depuis la connexion Mongoose existante (`connection.getClient().db()`) pour n'avoir qu'une seule connexion MongoDB dans toute l'application.

6.5 Deux bugs de configuration détectés et corrigés en cours de route

- Le rate-limiter global lisait des valeurs codées en dur au lieu des variables d'environnement prévues depuis la Phase 1 — corrigé.
- Les limites d'upload (`AUDIT_MAX_FILE_SIZE_BYTES` , etc.) étaient lues depuis `process.env` au chargement du module ES, **avant** que `dotenv` ait eu le temps de charger `.env` — un piège spécifique aux applications ESM avec NestJS. Corrigé en passant par `MulterModule.registerAsync()` , qui diffère la lecture jusqu'à ce que l'injection de dépendances soit prête.

7. Audit de sécurité complet

7.1 Points forts

Domaine	Constat
Upload de fichiers	Jamais écrit sur disque → élimine la classe de vulnérabilités "path traversal → écriture arbitraire"

Domaine	Constat
Authentification	Déléguée à Better Auth (bibliothèque dédiée, maintenue) plutôt que réimplémentée à la main
Autorisation	GET /audit/:id renvoie 404 (jamais 403) pour un rapport d'un autre utilisateur — n'expose pas son existence
Sortie IA	Double validation (schéma structuré Gemini + <code>class-validator</code> côté serveur) — aucune confiance aveugle envers le LLM
Secrets	Clé Gemini et token d'auth toujours en en-tête HTTP, jamais dans une URL ou un log
Erreurs	Le filtre global ne laisse jamais fuir de stack trace ou de chemin serveur vers le client
Confinement de l'IA	Gemini n'a aucune capacité d'action (pas de fonction calling, pas d'accès outils) — au pire un rapport erroné, jamais une action non désirée
Injection de prompt	Le prompt système instruit explicitement le modèle à traiter le contenu des fichiers comme des données, jamais comme des instructions
Défense en profondeur	Limites vérifiées à plusieurs niveaux (Multer + service applicatif)

7.2 Risques identifiés

[MOYEN] Rate-limiting incohérent en mode cluster — Sévérité moyenne

`express-rate-limit` stocke ses compteurs **en mémoire, par processus**. Avec `CLUSTERING_ENABLED=true` (plusieurs workers), chaque worker a son propre compteur — la limite réelle effective peut atteindre $N \times$ la valeur configurée selon la répartition des requêtes par l'OS. **Recommandation** : un store partagé (Redis, via `rate-limit-redis`) avant d'activer le cluster en production.

[MOYEN] Pas de vérification d'email — Sévérité faible à moyenne

`emailAndPassword: { enabled: true }` sans `requireEmailVerification` — un compte est utilisable immédiatement après inscription, sans confirmer la possession de l'adresse email. Acceptable en développement (aucun service SMTP n'est configuré), mais à revoir avant une mise en production ouverte au public.

[MOYEN] Pas de limite de complexité/format sur le mot de passe imposée explicitement — Sévérité faible

Better Auth applique ses règles par défaut (longueur minimale) ; aucune politique de mot de passe renforcée (complexité, liste de mots de passe compromis) n'a été configurée explicitement.

[FAIBLE] `CORS credentials: true` sans usage de cookies de session actuellement — Sévérité négligeable

Activé par anticipation pour le futur frontend ; sans authentification par cookie utilisée aujourd'hui (le bearer token est privilégié), ce n'est pas exploitable en l'état.

[FAIBLE] Pas de rotation/expiration explicite des sessions documentée — Sévérité faible

Better Auth gère l'expiration des sessions avec ses valeurs par défaut ; aucune politique spécifique au projet (durée de vie courte, refresh token, révocation) n'a été définie.

[OK] Prompt injection — Résiduel, accepté

Un fichier malveillant contenant du texte ressemblant à des instructions pourrait tenter de manipuler le modèle. Mitigé par le prompt système et sans impact grave car l'IA n'a aucune capacité d'exécution.

7.3 Ce qu'il reste à faire avant une exposition publique large

- 1. Vérification d'email (nécessite un service d'envoi d'emails)
- 2. Rate-limiting distribué si le mode cluster est activé en production
- 3. Politique de mot de passe et de session plus stricte
- 4. Frontend (aucune interface utilisateur n'existe encore)
- 5. Monitoring/alerting (aucun système de logs centralisé ou d'observabilité configuré)

8. Référence des variables d'environnement

Variable	Rôle	Exemple
PORT	Port d'écoute du serveur	3000
CLUSTERING_ENABLED	Active le mode multi-cœurs	false
CORS_ALLOWED_ORIGINS	Origines autorisées (séparées par virgules)	http://localhost:3001
GEMINI_API_KEY	Clé API Google Gemini	*(secret)*
GEMINI_API_ENDPOINT	Base URL de l'API Gemini	https://generativelanguage.googleapis.com/v1beta
GEMINI_MODEL	Modèle utilisé	gemini-flash-latest
GEMINI_TIMEOUT_MS	Timeout par appel	60000
GEMINI_MAX_RETRIES	Tentatives sur erreur transitoire	2
RATE_LIMIT_WINDOW_MS / RATE_LIMIT_MAX_REQUESTS	Limiteur global	900000 / 100
AUDIT_RATE_LIMIT_WINDOW_MS / AUDIT_RATE_LIMIT_MAX_REQUESTS	Limiteur dédié à /audit	900000 / 10
AUDIT_MAX_FILE_SIZE_BYTES	Taille max par fichier	2097152 (2 Mo)
AUDIT_MAX_FILES_PER_REQUEST	Nombre max de fichiers par requête	50
AUDIT_MAX_TOTAL_PAYLOAD_BYTES	Taille agrégée max	20971520 (20 Mo)
MONGODB_URI	Chaîne de connexion MongoDB	*(secret)*
BETTER_AUTH_SECRET	Secret de signature des sessions	*(secret, généré aléatoirement)*
BETTER_AUTH_URL	URL propre du backend (pas celle du frontend)	http://localhost:3000

9. Lancer le backend en local

```
cd backend
cp .env.example .env    # puis remplir GEMINI_API_KEY, MONGODB_URI, BETTER_AUTH_SECRET
pnpm install
pnpm run start:dev
```

Documentation interactive : <http://localhost:3000/api/docs>